NeuroKinematics — Motion Planning & Collision-Aware Path Layer

Roadmap Konumu

Bu aşama **Studio fazı** içerisinde uygulanacaktır.
Başlangıç koşulu:

- Foundations tamamlanmış olmalı.
- Core tamamlanmış olmalı.
- Hybrid solver altyapısı kullanılabilir durumda olmalı.
- Studio için robot modelleme/visualization altyapısı mevcut olmalı.
- **Joint-Space Trajectory Generation & Motion Execution Layer tamamlanmış ve PASS almış olmalı.**

Roadmap sırası:

Foundations

↓

Core

↓

Hybrid

↓

Studio

↓

Joint-Space Trajectory Generation

↓

Motion Planning & Collision-Aware Path Layer — BU AŞAMA

↓

Studio Interactive Simulation

↓

ROS2 / Physical Robot Integration

Önemli mimari ayrım:

Motion Planning modülü trajectory generator'dan sonra geliştirilecek olsa da sistemin runtime veri akışında trajectory

generator'dan önce çalışacaktır:

Target Pose

↓

IK Solver

↓

Target Joint Configuration

↓

Motion Planner

↓

Collision-Free Joint Path

↓

Trajectory Generator

↓

Time-Parameterized Trajectory

↓

Simulation / Execution

---

1. Amaç

NeuroKinematics sistemine robotun mevcut konfigürasyonundan hedef konfigürasyona yalnızca interpolasyon yaparak değil, çevredeki engelleri ve robotun kendi geometrisini dikkate alarak **collision-free bir yol bulmasını sağlayan motion planning katmanı** ekle.

Temel problem:

$q_{start} \rightarrow q_{goal}$

verildiğinde doğrudan interpolation yapmak yerine:

q_{start}

↓

q_1

↓

q_2

↓

...

↓

q_goal

şeklinde geçerli bir konfigürasyon uzayı yolu üret.
Her ara konfigürasyon:

- joint limitlerine uygun,
- çevresel çarpışmalardan uzak,
- mümkünse self-collision içermeyen,
- mevcut robot modeliyle tutarlı

olmalıdır.

---

2. Mimari Sınırlar

Bu aşama:

- IK solver değildir.
- trajectory generator değildir.
- motor controller değildir.

- dynamics solver değildir.

Motion planner yalnızca:
collision-free geometric path
üretmelidir.
Üretilen yol daha sonra mevcut trajectory modülüne verilerek:
q(t), dq(t), ddq(t)
haline dönüştürülmelidir.
Katmanları birbirine karıştırma.

---

3. Ana Runtime Akışı

Kullanıcı Studio üzerinde:

1. Robot seçer.
2. Başlangıç durumu belirler.
3. Hedef Cartesian pose belirler.

4. NeuroKinematics IK hedef joint configuration üretir.
5. Motion planner start ve goal arasında geçerli yol arar.
6. Collision checker yolu doğrular.
7. Path optimizer gereksiz waypointleri azaltır.
8. Trajectory generator yolu zaman-parametreli hale getirir.
9. Robot simülasyonda hareket eder.

Akış:

Current Joint State

↓

Target Cartesian Pose

↓

NeuroKinematics IK / Hybrid Solver

↓

Goal Joint Configuration

↓

Motion Planning

↓

Collision-Free Joint Path

↓

Path Simplification

↓

Trajectory Generation

↓

Trajectory Validation

↓

Studio Playback

---

4. Scene Representation

Motion planner robot dışında çevreyi de bilmelidir.
Merkezi bir PlanningScene veri modeli oluştur.
Örneğin:

PlanningScene

- robot_model
- robot_state
- obstacles
- allowed_collision_pairs
- workspace_bounds
- metadata

İlk sürümde desteklenecek obstacle primitive'leri:

- box
- sphere
- cylinder

Her obstacle için:

- unique ID
- geometry
- pose
- dimensions
- enabled/disabled

tutulmalıdır.
Örnek:
Box(id="table", position=[0.5, 0.0, 0.2], size=[0.8, 1.2, 0.4])
Scene modeli GUI'den bağımsız olmalıdır.

---

5. Robot Collision Geometry

Robotun collision geometry bilgisi mümkünse URDF üzerinden okunmalıdır.
Visual mesh ile collision mesh'i birbirinden ayır.
Öncelik sırası:

1. URDF collision geometry
2. simplified collision primitives
3. fallback bounding volumes

Planner render mesh üzerinden collision

hesaplamak zorunda bırakılmamalıdır.
Collision geometry mümkün olduğunca hafif tutulmalıdır.

---

6. Collision Checking

Aşağıdaki kontroller ayrı modüller olarak uygulanmalıdır:

6.1 Environment Collision

Robot linkleri ile scene obstacle'ları arasındaki çarpışmaları kontrol et.
Fonksiyon:
is_state_collision_free(q)
gibi merkezi bir API sağlamalıdır.

6.2 Self Collision

Uygulanabilir ise robot linkleri arasındaki self-collision kontrolünü ekle.
Komşu ve fiziksel olarak sürekli temas eden link çiftleri allowed-collision listesinde tutulmalıdır.
Örneğin:
link_1 ↔ link_2
gibi bağlantılar yanlış collision üretmemelidir.

6.3 Edge Collision

Sadece waypoint'leri kontrol etmek yeterli değildir.
q_A ile q_B arasındaki interpolasyon da collision açısından örneklenmelidir.
Fonksiyon:

is_edge_collision_free(q_A, q_B)
oluştur.
Resolution configurable olmalıdır.

---

7. Configuration Space

Planner robotu Cartesian XYZ uzayında değil öncelikle **joint configuration space** içerisinde planlamalıdır.
6DOF robot için:
C-space = [q1, q2, q3, q4, q5, q6]
Her joint sınırı robot modelinden alınmalıdır:
$q_{min} \leq q \leq q_{max}$
Sampling yalnızca geçerli joint limitleri içinde yapılmalıdır.

---

8. İlk Planning Algoritması

İlk gerçek planner olarak:
RRT-Connect
uygula.
Sebep:

- manipülatör planning problemlerinde uygundur,
- bidirectional arama kullanır,
- basit RRT'ye göre genellikle daha hızlı bağlantı kurar,
- NeuroKinematics için araştırma bağımsız baseline sağlar.

Planner interface algoritmadan bağımsız olmalıdır.
Örneğin:
MotionPlanner

plan(start_q, goal_q, scene, config)
çıktısı:
PlanningResult
olmalıdır.

---

9. PlanningResult

Standart çıktı oluştur.
PlanningResult:

- success
- path
- planning_time
- sampled_states
- collision_checks
- path_length
- planner_name
- random_seed

- failure_reason
- metadata

Path:
[q_start, q1, q2, ..., q_goal]
şeklinde olmalıdır.
Planner başarısız olduğunda boş path üretmek yerine anlamlı failure reason sağlamalıdır.
Örnek:
GOAL_IN_COLLISION
START_IN_COLLISION
NO_PATH_FOUND
TIMEOUT
INVALID_JOINT_LIMIT

---

10. Determinism

Araştırma/reproducibility gereksinimleri

nedeniyle planner configurable random seed kullanmalıdır.
Aynı:

- robot
- scene
- start
- goal
- planner parameters
- seed

ile mümkün olduğunca aynı planning sonucu üretilebilmelidir.
Seed tüm artifact ve loglarda saklanmalıdır.

---

11. Direct Path Test

Planner çalıştırılmadan önce:

start_q → goal_q
doğrudan edge collision-free mi kontrol edilmelidir.
Eğer doğrudan yol güvenliyse gereksiz planning yapılmamalıdır.
Akış:
Start ↓ Direct edge valid? ├── YES → direct path
└── NO → RRT-Connect
Bu basit senaryolarda ciddi performans kazancı sağlar.

---

12. Path Simplification

RRT tarafından üretilen path genellikle gereğinden fazla waypoint içerir.
Bu nedenle bir path simplification aşaması ekle.
Örneğin random shortcutting veya deterministic

shortcutting kullanılabilir.
Önce:
q0 → q1 → q2 → q3 → q4 → q5
sonra eğer:
q0 → q3
collision-free ise:
q1 ve q2 kaldırılabilsin.
Sonuç:
q0 → q3 → q5
gibi daha kısa yol üretilebilsin.
Raw path ve simplified path ayrı tutulmalıdır.

---

13. Path Metrics

Her planning işlemi için en az şu metrikleri hesapla:

- planning time

- success/failure
- number of sampled states
- number of collision checks
- raw waypoint count
- simplified waypoint count
- raw path length
- simplified path length
- minimum obstacle clearance mümkünse
- direct path available / unavailable

Joint-space path length:
Σ ||q_i+1 - q_i||
ile ölçülebilir.

---

14. Trajectory Layer Entegrasyonu

Motion planner zaman-parametreli trajectory üretmemelidir.

Planning başarılı olduktan sonra:
PlanningResult.path
mevcut trajectory generator'a aktarılmalıdır.
Örneğin:
planning_result = planner.plan(...)
trajectory =
trajectory_generator.generate_from_waypoints(
planning_result.path)
Trajectory layer:

- velocity limit
- acceleration limit
- interpolation
- timestamps

işlerinden sorumludur.
Motion planner bunları tekrar implement etmemelidir.

---

15. IK Entegrasyonu

Cartesian hedef doğrudan motion planner'a verilmemelidir.
Önce NeuroKinematics solver çalışmalıdır:
q_goal = nk_solver.solve(current_q, target_pose)
Ardından:
planner.plan(q_start=current_q, q_goal=q_goal)
çalışmalıdır.
Birden fazla geçerli IK çözümü varsa daha ileri versiyonda planner-aware IK selection desteklenebilecek şekilde architecture hazırlanmalıdır.
İlk sürümde bir solver çıktısı yeterlidir.

---

16. Start/Goal Validation

Planning öncesinde:
start_q
ve
goal_q
için:

- joint limit
- environment collision
- self collision

kontrollerini yap.
Geçersiz başlangıç veya hedef varsa planner çalıştırma.
Açık failure result döndür.

---

17. KUKA Demonstration Scenario

KUKA KR6 R900 sixx ile deterministik

demonstration oluştur.
Scene:

- KUKA robot
- robot önünde bir obstacle
- obstacle'ın arkasında erişilebilir target pose

Doğrudan:
q_start → q_goal
interpolasyonu obstacle ile çarpışmalıdır.
Motion planner ise obstacle'dan kaçan alternatif bir joint-space path bulmalıdır.
Gösterilecek senaryo:
START ↓ Straight trajectory ↓ COLLISION ❌
START ↓ RRT-Connect ↓ Collision-Free Path ↓ Trajectory Generation ↓ TARGET ✅
Bu demonstration aşamanın temel görsel kanıtı olmalıdır.

---

18. İkinci Demonstration

Multi-goal senaryosu oluştur:

Current Position

↓

Point A

↓

Point B

Her segment için:

IK

↓

Motion Planning

↓

Trajectory Generation

uygula.

A ve B arasında obstacle varsa planner uygun alternatif path üretmelidir.

---

19. Studio Integration API

GUI doğrudan RRT implementasyonuna bağımlı olmamalıdır.
Basit servis/API katmanı oluştur:
PlanningService
Önerilen fonksiyonlar:
set_robot()
set_scene()
add_obstacle()
remove_obstacle()
clear_scene()
set_start_state()
set_target_pose()
solve_ik()
plan()
validate_path()
generate_trajectory()
get_result()

Bu interface ileride Studio tarafından kullanılacaktır.

---

20. Visualization Support

Planner'ın ürettiği path visualizer'a aktarılabilmelidir.
Studio tarafında ileride:

- start configuration
- goal configuration
- raw path
- simplified path
- end-effector trace
- obstacles

görüntülenebilmelidir.
Bu aşamada tam GUI zorunlu değildir.

Ancak visualization-ready output zorunludur.

---

21. Debug Visualization

Development/test artifact olarak en az şu görseller oluştur:

1. Planning scene 3D görünümü
2. Start robot configuration
3. Goal robot configuration
4. Straight path collision örneği
5. Planned collision-free path
6. Raw vs simplified path
7. End-effector Cartesian trace

---

22. Benchmark Suite

En az üç planning difficulty seviyesi oluştur.

Easy

Obstacle yok veya doğrudan yol açık.

Medium

Tek obstacle direct path'i engelliyor.

Hard

Birden fazla obstacle veya dar geçiş bulunuyor.
Her scenario için belirli sayıda deterministik seed çalıştır.
Raporla:

- success rate
- median planning time
- P95 planning time

- path length
- sampled states
- collision checks

---

23. Acceptance Tests

T-MP01 — Direct path

Obstacle olmayan durumda direct path bulunmalı.

T-MP02 — Start collision

Start collision içerisindeyse planner hata vermeli.

T-MP03 — Goal collision

Goal collision içerisindeyse planner hata vermeli.

T-MP04 — Obstacle avoidance

Doğrudan yol çarpışırken planner collision-free alternatif yol üretmeli.

T-MP05 — Edge validation

Path üzerindeki hiçbir segment collision içermemeli.

T-MP06 — Joint limits

Tüm waypointler joint limits içerisinde olmalı.

T-MP07 — Self collision

Destekleniyorsa hiçbir waypoint self-collision içermemeli.

T-MP08 — Determinism

Aynı seed aynı veya eşdeğer deterministik

sonucu üretmeli.

T-MP09 — Path simplification

Simplification sonrasında path collision-free kalmalı.

T-MP10 — Trajectory integration

Planner path'i trajectory generator tarafından başarıyla zaman-parametreli hale getirilmeli.

T-MP11 — Final pose

Trajectory sonunda FK çıktısı hedef IK configuration ile eşleşmeli.

T-MP12 — Regression

Foundations, Core, Hybrid ve önceki Studio/Trajectory testleri bozulmamalı.

---

24. Mimari

Önerilen yapı:
src/neurokinematics/ planning/ __init__.py base.py scene.py collision.py state_validator.py edge_validator.py result.py service.py planners/ __init__.py rrt_connect.py optimization/ __init__.py shortcut.py metrics.py
Mevcut proje mimarisine uyum sağlamak için isimleri gerektiğinde adapte et.
Robot modeli, joint limitleri veya FK implementasyonunu tekrar oluşturma.
Mevcut NeuroKinematics kaynaklarını kullan.

---

25. Gelecek Genişlemelerine Hazırlık

İlk sürümde yalnızca RRT-Connect implement edilse bile interface gelecekte:

- RRT*
- PRM
- BIT*
- informed RRT*
- Cartesian planning
- constraint-aware planning
- MoveIt/OMPL adapter

eklenebilmesini desteklemelidir.
Planner interface'i algoritmaya hard-code edilmemelidir.

---

26. MoveIt / OMPL

İlk implementasyonda MoveIt zorunlu dependency haline getirilmemelidir.
Ama ileride karşılaştırma amacıyla:
NeuroKinematics Planner
vs
MoveIt/OMPL
benchmarkı yapılabilmesine uygun bir adapter mimarisi bırak.
Bu akademik karşılaştırma için değerlidir.

---

27. Bu Aşamada Yapılmaması Gerekenler

Scope creep oluşturma.
Aşağıdakileri bu aşamada uygulama:

- fiziksel robot kontrolü
- motor driver
- torque control
- inverse dynamics
- visual SLAM
- perception
- kamera tabanlı obstacle detection
- dynamic obstacle avoidance
- reinforcement learning planner
- grasp planning
- task planning
- ROS2 hardware bridge

Bu özellikler sonraki katmanlara bırakılmalıdır.

---

28. Dynamic Obstacles

İlk sürüm statik planning scene varsaymalıdır.
Hareket eden insanlar, araçlar veya dynamic obstacles kapsam dışıdır.
Bunlar gelecekte:
Dynamic Motion Planning / Replanning Layer olarak değerlendirilebilir.

---

29. Safety Terminology

Simulation'da collision-free path bulunması fiziksel robot açısından otomatik olarak "safe motion" anlamına gelmez.
Dokümantasyonda açık şekilde:
“Collision-free simulation path”
ifadesini kullan.
Fiziksel safety doğrulaması yapılmadan:
“Safe for physical execution”

iddiasında bulunma.

---

30. Dokümantasyon Güncellemesi

README ve Roadmap'i güncelle.
Mimari:
Target Pose ↓ NeuroKinematics IK ↓ Target Joint Configuration ↓ Motion Planning ↓ Collision-Free Path ↓ Trajectory Generation ↓ Time-Parameterized Trajectory ↓ Studio Simulation ↓ Future ROS2 / Physical Robot Bridge
şeklinde açıkça gösterilmelidir.
Ayrıca üç kavramı özellikle ayır:

IK

“Hedef uç-efektör pozu için joint configuration bulur.”

Motion Planning

“Başlangıç ve hedef joint konfigürasyonları arasında collision-free geometrik yol bulur.”

Trajectory Generation

“Bulunan geometrik yolu velocity/acceleration limitlerine uygun zaman-parametreli harekete dönüştürür.”

---

31. Git Süreci

Başlamadan önce:
git status
git branch
git log -1
kontrol et.

Trajectory layer'ın PASS durumunu doğrula.
İlgisiz dosyaları değiştirme.
Mantıksal commitlere ayır:

1. planning scene + collision API
2. state/edge validation
3. RRT-Connect
4. path simplification
5. trajectory integration
6. benchmark + demos
7. documentation

Her önemli adımda testleri çalıştır.

---

32. Nihai Kapanış Raporu

Çalışma sonunda aşağıdaki formatta rapor üret:

Status

PASS / PARTIAL / FAIL

Roadmap Position

Trajectory Layer sonrası Studio Motion Planning aşaması.

Architecture

Implemented Components

Collision System

Planner

Tests

Test	Status	Result

Benchmark

Scenario	Success Rate	Median Time	P95	Path Length

KUKA Demonstration

Straight Path: COLLISION / VALID

Planned Path: COLLISION-FREE / FAILED

Trajectory Integration

Generated Artifacts

Known Limitations

Regression Results

Git

Recommended Next Step

---

33. PASS Kriteri

Bu aşamaya yalnızca aşağıdakilerin tamamı sağlanırsa PASS ver:

- Planning scene çalışıyor.
- Robot-obstacle collision checking çalışıyor.
- Start/goal validation çalışıyor.
- Edge validation çalışıyor.
- RRT-Connect start → goal path bulabiliyor.
- Joint limits korunuyor.
- Path'in tamamı collision-free olarak doğrulanıyor.

- Path simplification sonucu geçerli kalıyor.
- Planner çıktısı trajectory layer'a aktarılabiliyor.
- Trajectory limitleri koruyor.
- Final configuration doğru.
- KUKA obstacle demonstration başarıyla tamamlanıyor.
- Benchmark artifactları üretiliyor.
- Mevcut test suite regression vermiyor.

Herhangi bir temel validation başarısızsa PASS verme.
Başarısızlıkları gizleme veya geçici workaround ile kabul kriterlerini atlama.